Projektuppgift

Programmering i C#.NET

Ekonomikollen

Kalle Persson



Mittuniversitetet

MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Kalle Persson, kape2002@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

Sammanfattning

Det här projektet handlar om att utveckla en enkel applikation för att kunna hålla reda på sina inkomster och utgifter. Min applikation heter Ekonomikollen och den är skapad i programmeringsspråket C# med .NET. Programmet körs i en konsol där användaren kan välja mellan olika funktioner genom en numrerad meny.

Du kan välja att lägga till inkomster eller utgifter, visa alla dina transaktioner, ta bort en registrering samt söka efter en transaktion. Programmet kan även räkna ut ditt saldo, din totala inkomst eller utgifter. All information sparas i en JSON-fil som också läses in vid uppstart av programmet, vilket gör att alla transaktioner finns kvar även efter att du avslutat programmet.

Arbetet är uppdelat i en datamodell, en fil som hanterar menyn samt en serviceklass, och vanliga fel vid inmatning kontrolleras med bland annat TryParse.

Applikationen har felsökts och testats genom att registrera olika belopp, ta bort transaktioner, söka på befintliga transaktioner samt omstart av programmet för att dubbelkolla så att informationen blir korrekt sparad.

Resultatet har blivit en fungerande grundversion av ett verktyg för personlig ekonomisk översikt. Det visar även hur flera delar av kursen kan användas tillsammans som exempelvis, klasser, listor, metoder, villkor, filhantering och LINQ bland annat.

Innehållsförteckning

Sammanfattning....iii

1 Introduktion....v
1.1 Bakgrund....v
1.2 Problemdefinition....v
1.3 Syfte och mål....v
1.4 Avgränsningar....vi

2 Teori....vii
2.1 Programmeringsspråket C#....vii
2.2 .NET och konsolapplikationer....vii
2.3 Objektorienterad programmering....vii
2.4 Listor och LINQ....viii
2.5 JSON och filhantering....viii
2.6 Inmatningskontroll och felhantering....viii

3 Metod....ix
3.1 Arbetsmetod....ix
3.2 Verktyg och utvecklingsmiljö....ix
3.3 Planering och uppdelning....ix
3.4 Testmetod....x

4 Konstruktion....x
4.1 Programmets struktur....x
4.2 Datamodellen Transaction....xi
4.3 Huvudmenyn....xi
4.4 Flödesschema....xii
4.5 Registrering av inkomster och utgifter....xiii
4.6 Beräkning av ekonomisk sammanställning....xiii
4.7 Sökning och borttagning....xiv
4.8 Sparande och inläsning av JSON....xiv

5 Resultat och testning....xv
5.1 Färdig applikation....xv
5.2 Exempel på programmets utdata....xv
5.3 Genomförda tester....xvi
5.4 Koppling mellan mål och resultatet....xvi

6 Slutsatser....xviii
6.1 Vad fungerade bra?....xviii
6.2 Problem under utvecklingen....xviii
6.3 Begränsningar och förbättringar....xviii
6.4 Vad jag har lärt mig....xix

Källförteckning....xx

1 Introduktion

1.1 Bakgrund

Under kursens gång så har flera viktiga delar av programmering behandlats. För mig så är projektuppgiften en möjlighet till att använda kunskaperna i ett större program där olika funktioner samarbetar. Jag har valt att utveckla ett enkelt ekonomiprogram därför att jag tycker att det har ett tydligt användningsområde och går att bygga med de tekniker som ingått i kursen.

Det kan ibland vara svårt att komma ihåg den inkomst samt utgifter man har, speciellt om det är flera små utgifter som registreras vid olika tillfällen över en längre period. Om man inte har en tydlig överblick på sina pengar kan det också bli svårt att veta hur mycket pengar man faktiskt har kvar. Genom att samla både inkomster och utgifter i ett enkelt program så blir det lättare att följa hur ekonomin förändras över tid.

1.2 Problemdefinition

Problemet som ska lösas är hur en användare kan registrera och följa upp vanliga ekonomiska transaktioner i en lokal applikation. Om alla uppgifter bara finns i programmets minne så försvinner de när programmet avslutats, så därför måste det också kunna sparas och läsas in igen. Programmet behöver också kunna hantera en felaktig inmatning och ge en tydlig återkoppling.

Projektet fokuserar på manuellt registrerade utgifter och inkomster, där det även ska vara möjligt att se alla poster, räkna ut ditt saldo, söka efter transaktioner samt möjlighet till att ta bort poster.

1.3 Syfte och mål

Mitt syfte är att skapa en användbar konsolapplikation i C# och .NET samt att visa hur teori från kursen kan användas i praktiken. Målet med applikationen är att den ska fungera från start till avslut och att det ska finnas sparade transaktioner kvar även när programmet startas om på nytt.

Dessa är några av de mål jag har satt på vad programmet ska kunna göra:

- Kunna registrera inkomster och utgifter med ett belopp, en kategori samt en beskrivning.
- Ge en överblick över alla registrerade transaktioner med ett specifikt ID och datum.
- Kunna ta bort en transaktion med hjälp av dess ID.
- Beräkna summan av dina inkomster och utgifter samt visa ditt nuvarande saldo.
- Kunna söka efter transaktioner med hjälp av ord ifrån kategorin eller beskrivningen.
- Spara och läsa in transaktioner från en JSON-fil.
- Kontrollera flera vanliga typer av felaktig inmatning.

1.4 Avgränsningar

Projektet är inte något komplett bokföringssystem som är avsett för att hantera känsliga bankuppgifter, utan istället ett lokalt konsolprogram. Det finns ingen databas, någon webbplats eller användarkonton. Som användare så registrerar du transaktionerna manuellt och datumet sätts automatiskt till dagens datum.

2 Teori

2.1 Programmeringsspråket C#

C# är ett programmeringsspråk som kan användas till webbapplikationer, konsolprogram och andra typer av program. I mitt projekt så har C# använts för bland annat att läsa in text till en användare, utföra beräkningar samt styra vilka funktioner som körs. C# har stöd för metoder, klasser och olika datatyper. [1]

I projektet använder jag mig också av exempelvis string för text, int för ID-nummer, DateTime för datum och decimal för belopp. Decimal är passande när det är belopp som ska hanteras eftersom att datatypen är avsedd för decimaltal där noggrannheten är extra viktig. Variabler används också för att spara värden. [1, 7]

2.2 .NET och konsolapplikationer

.NET är en utvecklingsplattform innehållandes verktyg och ett bibliotek för att kunna bygga och köra applikationer. Mitt projekt skapades som en konsolapplikation med kommandot ”dotnet new console” och körs via ”dotnet run”. Konsolen används för att visa en meny samt ta emot användarens val. [3]

Utifrån projektets storlek så passar en konsolapplikation bra eftersom att den inte kräver att ett grafiskt användargränssnitt byggs. Det underlättar för att kunna fokusera mer på programmets logik och filhantering, men med den nackdelen att användaren måste skriva in alla uppgifter själv.

2.3 Objektorienterad programmering

En objektorienterad programmering innebär att ett program kan delas upp i klasser som har olika ansvar. I mitt projekt så används klassen Transaction för beskrivningen av en ekonomisk post. Klassen EconomyService innehåller metoder som arbetar med posterna. En uppdelning gör så att koden blir lättare överskådlig istället för att allting skulle ligga i en och samma fil. [2]

Ett objekt av Transaction innehåller exempelvis ett ID, ett belopp, en typ, en kategori, ett datum och en beskrivning och flera objekt kan sedan sparas i en lista.

2.4 Listor och LINQ

List<Transaction> har använts för att lagra flera transaktioner medan programmet körs. När nya poster läggs till så växer listan, medan den minskar när poster tas bort. I mitt projekt så används även LINQ som gör det möjligt för att söka och filtrera objekt i en samling. [1]

Metoden FirstOrDefault har använts för att kunna hitta en transaktion med ett visst ID. Metoden Where har också använts för att hitta transaktioner vars beskrivning eller kategori innehåller sökordet, och sökningen använder StringComparison.OrdinalIgnoreCase som gör att stora eller små bokstäver inte behöver stämma överens.

2.5 JSON och filhantering

JSON är ett textbaserat format som används för att lagra information på ett strukturerat sätt. I mitt projekt använder jag System.Text.Json för att spara programmets transaktioner. När informationen ska sparas omvandlas listan med transaktioner till JSON med hjälp av JsonSerializer.-Serialize. När sedan programmet startas igen så används JsonSerializer.Deserialize för att läsa in informationen och omvandla tillbaka den till en lista med transaktioner. [4]

Klassen File används för att läsa och skriva till filen, medans File.Exists kontrollerar om filen finns. [4] Detta gör så att samma transaktioner kan användas vid en ny programstart. Mitt val av JSON var eftersom att formatet är lätt att förstå och det räckte för projektets begränsade datamängd.

2.6 Inmatningskontroll och felhantering

I ett konsolprogram är det viktigt att kontrollera vad användaren skriver in för att undvika fel. Skriver exempelvis användaren in en text där ett tal förväntas så måste programmet kunna hantera det. I projektet så används decimal.TryParse och int.TryParse för att kontrollera att inmatningen går att omvandla till ett tal. Det gör även en kontroll på att beloppet är större än 0 samt att användaren inte lämnar en kategori tom. [5, 6]

De här kontrollerna gör programmet enklare att använda och minskar risken för fel när användaren skriver in information. Det hindrar dock inte programmet från att stöta på andra problem, om exempelvis JSON-filen där alla transaktioner sparas skulle bli skadad så kan programmet få problem med att läsa in informationen. I min nuvarande version så finns det ingen särskild hantering för en sån situation.

3 Metod

3.1 Arbetsmetod

Jag genomförde arbetet stegvis där jag först bestämde vad programmet skulle kunna göra och efter det så skapade jag projektet och en datamodell för transaktioner. Jag fortsatte med att utveckla funktionerna och testkörde dem i konsolen och när mina grundfunktioner fungerade så adderade jag JSON för en fungerande inläsning och sparning. Sist så kompletterades programmet med borttagning och sökning.

Att göra allting stegvis gav mig en chans att upptäcka eventuella problem tidigt och varje ny funktion kunde kontrolleras innan en ny del adderades. Metoden kändes extra användbar när jag la in filhanteringen, för den behövde fungera tillsammans med både registrering och borttagning.

3.2 Verktyg och utvecklingsmiljö

Jag utvecklade min applikation i VSC (Visual Studio Code) på Windows. Programmet är skrivet i C# och använder standardbiblioteken i .NET. Jag har inga externa paket.

Det är ett konsolprojekt med en Program.cs-fil samt separata filer för programlogik och datamodell, separeringen gör det enklare att se vad som ansvarar för vad när det kommer till inmatning, beräkningar och lagring.

3.3 Planering och uppdelning

Innan jag började på att koda så delade jag upp lösningen i tre huvuddelar: användargränssnitt, datamodell samt funktioner där användargränssnittet skulle visa menyn och läsa användarens val. Beskrivning av en transaktion skulle komma ifrån datamodellen och serviceklassen skulle hantera listan, beräkningarna samt filen.

Jag använde en enkel skiss av huvudmenyn som utgångspunkt när jag planerade hur programmet skulle fungera. Menyn innehåller numrerade alternativ vilket gör att användaren kan välja mellan de olika funktioner som programmet har, på så sätt så kan samma huvudloop användas för att hantera flera olika val.

Del	Ansvar
Program.cs	Meny, användarinmatning och återkoppling
Transaction.cs	Beskriver vilka uppgifter en transaktion innehåller
EconomyService.cs	Hanterar transaktioner, beräkningar, JSON-lagring och sökningar

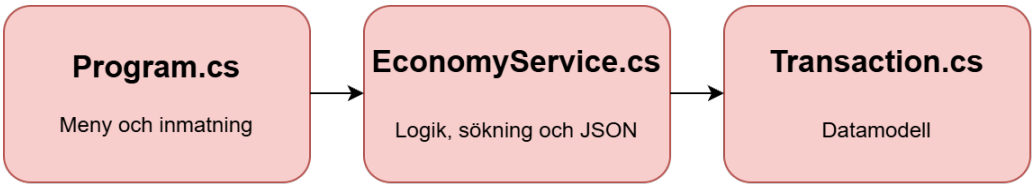
3.4 Testmetod

Jag gjorde en manuell testning där jag startade upp programmet och provade alla olika menyval, med både korrekta och felaktiga värden för att testa. Kontrollerade även att saldot blev rätt räknat och att sparade transaktioner fanns kvar efter en omstart av programmet.

4 Konstruktion

4.1 Programmets struktur

Det är tre centrala källkodsfiler som programmet består av. Program.cs styr menyvalen och anropar rätt metoder medans EconomyService.cs innehåller listan med transaktioner samt metoder för att visa, ändra och spara den. Transaction.cs är filen som beskriver vilka uppgifter en transaktion har.



Figur 1. Förenklad översikt över programmets tre huvuddelar.

4.2 Datamodellen Transaction

Varje transaktion representeras av ett objekt. ID används till exempel för att kunna skilja olika poster åt vid borttagning. Type anger om posten är en inkomst eller utgift. Amount är beloppet, Category vilken slags typ av kostnad eller inkomst det är och Description ger extra information. Date visar när posten blev registrerad.

```
4   public class Transaction
5   {
6       public int Id { get; set; }
7
8       public string Type { get; set; } = "";
9
10      public decimal Amount { get; set; }
11
12      public string Category { get; set; } = "";
13
14      public string Description { get; set; } = "";
15
16      public DateTime Date { get; set; }
17  }
```

Kodexempel 1. Egenskaper i klassen Transaction (förkortat till själva klassen).

I EconomyService så använder jag en lista av typen List<Transaction>, så när en ny post blir skapad så får den ett ID som är större än det högsta ID-numret i den aktuella listan. Skulle listan vara tom så börjar numreringen på 1.

4.3 Huvudmenyn

I programmet så använder jag en while-loop som fortsätter så länge variabeln running är true. Inuti loopen så visas huvudmenyn där användaren sedan får välja mellan programmets olika funktioner. Användarens val hanteras av en switch-sats som avgör vilken metod som ska köras och ifall användaren väljer alternativ 0 så avslutas loopen samt programmet. Vid ett ogiltigt menyval så visas ett felmeddelande.

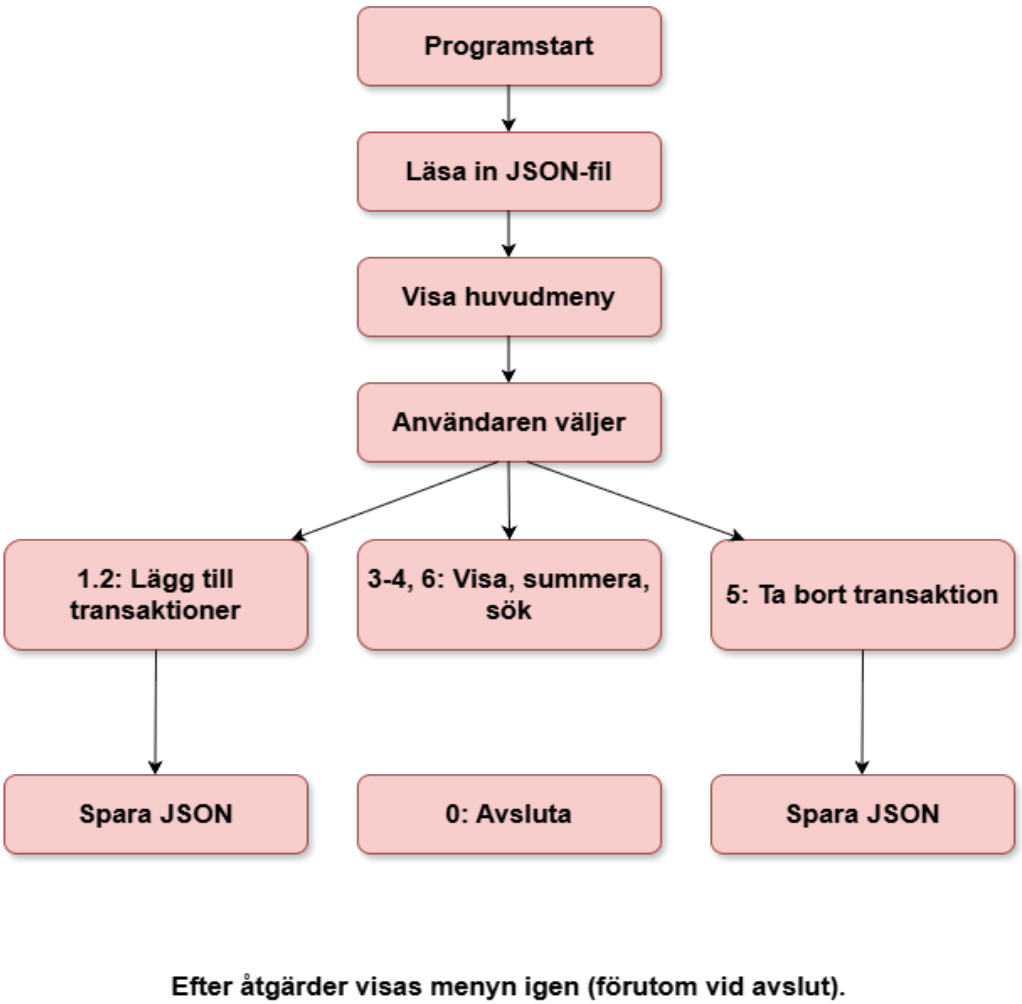
```
            case "0":
                running = false;
                break;

            default:
                Console.WriteLine("Felaktigt menyval. Försök igen.");
                Pause();
                break;
        }
    }
```

Kodexempel 2. En del av huvudmenyn i Program.cs där en while-loop och en switch-sats används för att hantera användarens val.

4.4 Flödesschema

Bilden nedan visar hur programmet startar, läser in tidigare transaktioner och visar huvudmenyn. En funktion utförs beroende på användarens val och när en post läggs till eller tas bort så sparas listan igen, och användaren återgår till menyn. Om användaren väljer menyval 0 avslutas programmet.



Figur 2. Ett flödesschema för applikationens huvudsakliga programflöde.

4.5 Registrering av inkomster och utgifter

När en användare väljer att lägga till en inkomst eller utgift så anropas samma metod med olika värde för typ. Det första som händer är att beloppet efterfrågas, programmet använder decimal.TryParse och kontrollerar sedan om beloppet är större än noll. Ifall kontrollen misslyckas så får användaren försöka igen, efter det så får användaren fylla i en kategori samt en beskrivning på antingen inkomsten eller utgiften och kategorifältet får inte lämnas tomt. När användaren har fyllt i uppgifterna så kommer datumet att sättas automatiskt med DateTime.Today.

Efter att uppgifterna är godkända så skickas de till EconomyService.AddTransaction. Där så skapas ett nytt Transaction-objekt som läggs till i listan. Efter att registreringen är klar så anropas Save för att skriva listan till JSON-filen.

```
        if (decimal.TryParse(Console.ReadLine(), out amount)
            && amount > 0)
        {
            break;
        }

        Console.WriteLine("Ange ett giltigt belopp större än 0.");
    }
```

Kodexempel 3. Kontroll av belopp vid registrering.

4.6 Beräkning av ekonomisk sammanställning

När programmet ska beräkna saldot så går den igenom alla transaktioner med en foreach-loop. Ifall Type är Inkomst så läggs beloppet till totalIncome, om Type är Utgift så läggs beloppet istället till totalExpenses. I slutet så räknas saldot fram genom att utgifterna dras ifrån inkomsterna.

```
    decimal balance = totalIncome - totalExpenses;

    Console.WriteLine("EKONOMISK SAMMANSTÄLLNING");
    Console.WriteLine("=========================");
    Console.WriteLine();

    Console.WriteLine($"Totala inkomster: {totalIncome:F2} kr");
    Console.WriteLine($"Totala utgifter:  {totalExpenses:F2} kr");
    Console.WriteLine("-------------------------");
    Console.WriteLine($"Kvarvarande saldo: {balance:F2} kr");
}
```

Kodexempel 4. Uträkning samt utskrift av saldot.

Ifall användaren exempelvis registrerar 5000 kronor i inkomst och 500 kronor i utgift så kommer saldot bli 4500 kronor. Beloppen skrivs ut med två decimaler.

4.7 Sökning och borttagning

För sökfunktionen så används LINQ-metoden Where för att gå igenom alla transaktioner. Det programmet gör är att den jämför sökordet med både Category och Description och jämförelsen tar inte någon hänsyn till om det används små eller stora bokstäver. Ifall inga poster matchas så får användaren ett meddelande.

När användaren väljer att ta bort en transaktion så visas först alla registrerade transaktioner tillsammans med deras ID-nummer. Användaren anger sedan ID-numret för den transaktion som ska tas bort. Programmet använder FirstOrDefault för att hitta rätt transaktion. Om den finns tas den bort från listan och JSON-filen uppdateras. Om ID-numret inte finns visas i stället ett meddelande, och ingen transaktion tas bort.

```
        Transaction? transaction = Transactions.FirstOrDefault(
            t => t.Id == id);

        if (transaction == null)
        {
            return false;
        }

        Transactions.Remove(transaction);

        Save();

        return true;
    }
```

Kodexempel 5. Huvudlogiken vid en borttagning av transaktion.

4.8 Sparande och inläsning av JSON

När en lista sparas så omvandlas den till en JSON-text, sen så skrivs den till filen transactions.json i programmets körningsmapp. Nästa start så kommer Load att kontrollera om filen finns och om den finns så kommer innehållet att läsas in och omvandlas till en lista med Transaction-objekt.

```
    string json = JsonSerializer.Serialize(
        Transactions,
        new JsonSerializerOptions { WriteIndented = true });

    File.WriteAllText(filePath, json);
}
```

Kodexempel 6. Ett sparande av transaktioner i JSON-format.

Filen görs lättare att läsa med WriteIndented och valet för JSON-formatet var för det faktum att det känns enkelt och inte kräver en databas. En begränsning är dock att programmet inte fångar upp fel om filen innehåller ogiltig JSON eller om disken inte går att skriva till.

5 Resultat och testning

5.1 Färdig applikation

Det slutgiltiga resultatet av projektet är en konsolapplikation med sex funktioner i huvudmenyn. Användaren kan lägga till inkomster och utgifter, visa registrerade transaktioner, få en överblick över sitt saldo, ta bort transaktioner och söka efter transaktioner i kategori samt beskrivning. Alla transaktioner sparas automatiskt och läses in på nytt när programmet startas.

Programmet visar ett ID, typ av belopp, en kategori och beskrivning för varje transaktion samt ett datum för när transaktionen lades till. Är listan helt tom så visas ett meddelande om att det inte finns några transaktioner.

5.2 Exempel på programmets utdata

```
========== EKONOMIKOLLEN ==========

1. Lägg till inkomst
2. Lägg till utgift
3. Visa alla transaktioner
4. Visa ekonomisk sammanställning
5. Ta bort transaktion
6. Sök transaktioner

0. Avsluta

Välj ett alternativ:
```

Figur 3. Ett exempel på hur huvudmenyn ser ut när programmet körs



Figur 4. Ett exempel på den ekonomiska sammanställningen.

5.3 Genomförda tester

Applikationen har testats manuellt i konsolen, och jag har sammanfattat de kontroller som jag genomfört under arbetet som även fungerade under testtillfället.

Test	Förväntat resultat	Utfall
Ogiltigt menyval	Felmeddelande, menyn fungerar	Godkänt
Negativt belopp	Beloppet avvisas	Godkänt
Text istället för ett belopp	Beloppet avvisas	Godkänt
Tom kategori	En ny inmatning krävs	Godkänt
Sökning med stora och små bokstäver	Rätt träff visas	Godkänt
Ta bort ett okänt ID	Meddelande utan krasch	Godkänt
Inkomst och utgift	Saldot uppdateras rätt	Godkänt
Starta om programmet	Sparade poster finns kvar	Godkänt
Ta bort en transaktion och starta om	Borttagningen finns kvar	Godkänt

5.4 Koppling mellan mål och resultatet

De målen som jag hade satt i avsnitt 1.3 uppnåddes efter tester i min version. Det går att registrera nya transaktioner, visa dem samt söka och ta bort dem. Programmet kan beräkna ett saldo samt sparar all information mellan körningarna. Det finns även kontroller för vanliga inmatningsfel.

6 Slutsatser

6.1 Vad fungerade bra?

Det som känns som det viktigaste resultatet är att applikationens alla olika delar fungerar tillsammans. En transaktion som blir registrerad kan visas i listan, påverkar det befintliga saldot och kan även hittas med sökfunktionen. Tar man bort transaktionen så uppdateras både listan och den sparade filen. Sen tycker jag också att uppdelningen i olika klasser gör att programmet blir lättare att läsa.

En nyckelfaktor för mig var att utveckla en funktion i taget, det gjorde att det gick att testa och rätta problem allt eftersom istället för att hela programmet skulle bli drabbat. Programmet blev även mer användbart med JSON-lagringen eftersom uppgifterna inte försvinner vid en avslutning.

6.2 Problem under utvecklingen

Ett par mindre problem, men ett var ett kompileringsfel i EconomyService.cs på grund av en saknad klammerparentes. Det löste sig relativt snabbt dock efter en grundläggande koll över koden.

Ett annat var att programfilen var låst när jag skulle starta en ny körning, och det var så enkelt som att min tidigare körning av programmet fortfarande var aktiv men jag tycker ändå det visar på hur viktigt det kan vara att läsa felmeddelanden noggrant.

6.3 Begränsningar och förbättringar

Programmet är rätt så enkelt, konsolens gränssnitt fungerar efter förväntningarna men det är inte lika lätt att överblicka som ett grafiskt program. Förbättringar i framtida versioner hade kunnat vara att visa en sammanställning per månad eller kategori, samt en möjlighet att redigera befintliga transaktioner och att kunna välja ett datum manuellt.

Tittar man på tekniska förbättringar så skulle automatiska enhetstester vara en bra förbättring, samt att lägga till en try/catch vid filhanteringen så att programmet skulle klara av att hantera en skadad JSON-fil eller eventuella problem med filbehörigheter. Sen skulle man kunna implementera ett bättre sätt att hantera ID-nummer över tid, just nu så kan man återanvända ett tidigare ID efter att den högsta posten tagits bort.

6.4 Vad jag har lärt mig

Jag tycker att projektet har gett mig en större praktisk erfarenhet när det kommer till att kombinera fler delar av C# och .NET i ett program. I projektet så har jag fått arbeta med klasser, listor, objekt, loopar, villkor, filhantering och metoder. Jag har fått en större förståelse för hur man kan kontrollera användarinmatning och även hur man kan dela upp programmet i mindre delar för att få en bättre arbetsstruktur.

Testning borde göras löpande och om jag hade gjort om arbetet så hade jag planerat fler tester tidigare och dokumenterat dem bättre för att lättare upptäcka eventuella problem.

Källförteckning

[1] Mittuniversitetet, Programmering i C#.NET DT071G
Föreläsning, Programmeringsspråket C# (2026-09-08)
https://play.miun.se/media/Mikael+Hasselmalms+Personal+Meeting+Room/0_zp1m9zj2
(Föreläsare Mikael Hasselmalm)
Hämtad 2026-10-31

[2] Mittuniversitetet, Programmering i C#.NET DT071G
Föreläsning, Objektorienterad programmering med C# (2026-09-02)
https://play.miun.se/media/C_OOP_HT26/0_jn7v365b
(Föreläsare Mikael Hasselmalm)
Hämtad 2026-10-31

[3] Mittuniversitetet, Programmering i C#.NET DT071G
Föreläsning, Presentation av kursen, Installera C# (2026-09-22)
https://play.miun.se/media/DT071G+-+Introduktion+HT26/0_buegukon
(Föreläsare Mikael Hasselmalm)
Hämtad 2026-10-31

[4] Mittuniversitetet, Programmering i C#.NET DT071G
Föreläsning, Filhantering/Serialisering (2026-09-28)
https://www.youtube.com/watch?v=JcuomBtBnGI
(Föreläsare Lars Lundin)
Hämtad 2026-10-31

[5] Microsoft (u.å.). Decimal.TryParse Method. Microsoft Learn.
Tillgänglig på:
https://learn.microsoft.com/en-us/dotnet/api/system.decimal.tryparse
Hämtad 2026-10-31

[6] Microsoft (u.å.). Int32.TryParse Method. Microsoft Learn.
Tillgänglig på:
https://learn.microsoft.com/en-us/dotnet/api/system.int32.tryparse
Hämtad 2026-10-31

[7] Microsoft (u.å.). Decimal Struct. Microsoft Learn.
Tillgänglig på:
https://learn.microsoft.com/en-us/dotnet/api/system.decimal
Hämtad 2026-10-31